altairsim — Quick Reference

2026-10-14 · 56b1c31

Contents

- Getting out, and back in
- Editing the command line
- Command line
- Monitor commands
- Boards
- Machines
- A machine file, in one look
- Endpoints — `CONNECT <id>:<unit> <endpoint>`

Getting out, and back in

Key	Does
`^E`	**STOP** — stop the machine and take the keyboard back. Nothing is lost.
`RUN`	Resume, at the exact instruction it stopped on.
`QUIT`	Leave. (There is no `EXIT`.)

Editing the command line

`Tab` completes what you are typing — a command, then a board id, then its property names, then a property's values (`SET mem0 fill=` then `Tab`); a second `Tab` lists the choices. `Up` / `Down` walk the command history, saved per directory in `.altairsim_history`.

Key	Does
`Ctrl-A` / `Ctrl-E` (or `Home` / `End`)	start / end of line
`Alt-B` / `Alt-F` (or `Ctrl-Left` / `Ctrl-Right`)	back / forward one word
`Ctrl-W` / `Ctrl-K` / `Ctrl-U`	erase word behind / to end of line / whole line
`Backspace` / `Delete`	erase before / under the cursor

Command line

```
altairsim [machine] [options]

  machine            a built-in name, or a config file (has a '/' or ends .toml).
                     Omitted: ./altairsim.toml if there is one, else `default`.
  -m, --machine <n>  ALWAYS a built-in name -- never a file.
  -f, --file <path>  ALWAYS a file -- never a built-in name.
  -n, --none         empty backplane: no boards, no memory, nothing.
  -l, --list         list the built-in machines and exit.
  -s, --script <f>   run a command script, then exit with its status. Paths in
                     it are relative to the script's folder.
  -x, --exec <cmd>   run one monitor command (repeatable), then exit.
  -i, --interactive  after --script/--exec, stay in the monitor.
      --set <text>   change a setting for this run, before the machine's startup
                     commands (repeatable). The text is what follows SET at the
                     prompt: --set "cpu0 clock_hz=2000000".
                     -s, -x, -i and --set cannot be used with --mcp.
      --mcp          MCP server on stdio.
      --mirror <sink>  with --mcp: mirror the console so a person can watch and
                     take over. socket:PORT (telnet in), or on macOS and Linux
                     pty or pty:LINK (open the link with a terminal program).
                     Add ?ro for watch-only.
      --log <file>   with --mcp: write what the guest prints to a text file, as
                     it prints it. An ordinary file; starts empty each run.
  -v, --version      print the version and exit.
  -h, --help         print this help and exit.
```

Monitor commands

Type the part before the bracket.

Command	Does	Usage
`BO[ARDS]`	List, add, or remove boards on the backplane.	`BOARDS [LIST]|ADD <type> <id> [k=v...]|REMOVE <id>`
`B[REAK]`	Set a breakpoint on an address, memory/I/O access, or tape stop.	`BREAK [<addr> | MEM R|W <addr> | IO R|W <port> | TAPE STOP] [IF <expr> | LOADS <expr>] [TRACE ON|OFF]`
`COM[PARE]`	Compare a range of memory against another address.	`COMPARE <range> <addr>`
`C[ONFIG]`	Load or save the whole machine as a TOML file.	`CONFIG LOAD <f.toml> | CONFIG SAVE <f.toml>`
`CONN[ECT]`	Attach a serial unit to an endpoint (console, socket, file, ...).	`CONNECT <id>:<u> <endpoint>`
`CONS[OLE]`	Show or set the host console's properties.	`CONSOLE [<k>=<v>...]`
`DE[POSIT]`	Write bytes into memory at an address.	`DEPOSIT <addr> <bytes...>`
`DI[SASM]`	Disassemble memory into instructions.	`DISASM [<addr>|<range>] [n] [CPU=8080]`
`DISC[ONNECT]`	Unplug the endpoint from a serial unit.	`DISCONNECT <id>:<u>`
`DO`	Run a file of monitor commands, one per line, as if typed.	`DO <file>`
`D[UMP]`	Show memory as hex and ASCII.	`DUMP [<addr>|<range>] [WIDTH=16]`
`E[DIT]`	Enter bytes into memory interactively from an address.	`EDIT <addr> [ROM]`
`EX[AMINE]`	Point the front panel at an address (and show that byte).	`EXAMINE [<addr>]`
`F[ILL]`	Fill a range of memory with a byte.	`FILL <range> <byte>`
`HE[LP]`	Show help for a command.	`HELP [<command>]`
`H[ISTORY]`	Replay the recent instruction (or bus-cycle) history.	`HISTORY [BUS|CPU] [n]`
`I[N]`	Read a byte from an I/O port.	`IN <port>`

`L[OAD]`	Load a file into memory (binary, Intel hex or S-record).	`LOAD <file> [AT <addr>] [FORMAT=BIN|HEX|SREC] [ROM]`
`MA[CHINE]`	Load a built-in machine by name (MACHINE none empties the backplane).	`MACHINE <name> | MACHINE none`
`M[OUNT]`	Put a disk or tape image into a drive; a .imd is converted to a raw .dsk beside it.	`MOUNT <id>[:<u>] <file> [WP] [CREATE] [extract[=<base>]] [k=v...]`
`MOV[E]`	Copy a range of memory to another address.	`MOVE <range> <dest> [ROM]`
`N[EXT]`	Step one instruction, running any CALL/RST to completion.	`NEXT`
`NO[BREAK]`	Remove a breakpoint, or all of them.	`NOBREAK [id]`
`NOP[ASTE]`	Stop a PASTE that is not finished.	`NOPASTE`

`O[UT]`	Write a byte to an I/O port.	`OUT <port> <byte>`
`PA[STE]`	Send a host file of any size to the guest's keyboard, as if pasted.	`PASTE <file>`
`P[OWER]`	Power-cycle the machine -- the only thing that clears RAM.	`POWER`
`Q[UIT]`	Leave the simulator.	`QUIT`
`REGI[ON]`	Add a RAM or ROM region to a memory board.	`REGION ADD <id> type=ram|rom at=<addr> [size=|mount=]`
`RE[GS]`	Show the CPU registers (SET REG changes one).	`REGS | SET REG <r>=<v>`
`RES[ET]`	Reset the machine, keeping RAM (RESET CPU resets just the processor).	`RESET [CPU]`
`REST[ORE]`	Load machine state back from a snapshot.	`RESTORE <file>`

`R[UN]`	Start or resume the machine, optionally at an address.	`RUN [addr]`
`SA[VE]`	Write a range of memory out to a file.	`SAVE <file> <range> [FORMAT=BIN|HEX|OCTAL|PRN]`
`SEA[RCH]`	Find bytes or a string in a range of memory.	`SEARCH <range> <bytes...>|"str"`
`SE[T]`	Change a property of a board, the console, display, a register, the bus, or the machine.	`SET <id>[:<u>]|CONSOLE|DISPLAY|TERMINAL|MACHINE|REG|BUS <k>=<v>`
`SH[OW]`	Display the state of a board, the bus, or the machine.	`SHOW <id>|BOARDS|BOARD <type> [UNITS]|MACHINES|MACHINE [<name>]|BUS [MAP|IO|IRQ|CONTENTION]|ROMS|MOUNTS|PATHS|CONSOLE|DISPLAY|TERMINAL|JOYSTICKS|DEBUG|SYMBOLS|CLO`
`SN[APSHOT]`	Save the whole machine state to a file.	`SNAPSHOT <file>`

`S[TEP]`	Run one instruction (or n), showing the registers after each.	`STEP [n]`
`SY[MBOLS]`	Load or clear a symbol table for disassembly.	`SYMBOLS LOAD <file> [REPLACE] | SYMBOLS CLEAR`
`T[RACE]`	Log every bus cycle while the machine runs.	`TRACE ON|OFF [file] [MASK=IN,OUT,IRQ,DMA,CONTENTION]`
`TY[PE]`	Feed text to the guest as if typed at its keyboard.	`TYPE "text"`
`U[NMOUNT]`	Take a disk or tape out of a drive.	`UNMOUNT <id>:<u>`
`W[HO]`	Say which board answers an address or I/O port.	`WHO <addr> | WHO IO <port>`

Boards

CPU

Type	What it is
`8080`	MITS 88-CPU: 8080A CPU board
`8085`	Generic 8085 CPU board
`z80`	Generic Z80 CPU board

Memory

Type	What it is
`bankmem`	Bank-switched RAM (Vector, Cromemco, North Star, ExpandoRAM)
`memory`	RAM/ROM board: plain, unbanked memory regions
`v2z80rom`	S100Computers V2 Z80: paged monitor EEPROM

Disk

Type	What it is
`16fdc`	Cromemco 16FDC: floppy controller + console UART, RDOS 2.52
`64fdc`	Cromemco 64FDC: floppy controller + console UART, RDOS 3.12
`dcdd`	MITS 88-DCDD: 8" hard-sector floppy controller
`dualide`	S100Computers IDE-AB: two CompactFlash sockets for CP/M 3
`dualsd`	S100Computers Dual SD: two microSD sockets for CP/M 3
`fdcplus`	FarmTek FDC+: serial drive (drive types 6, 7)
`hdsk`	MITS 88-HDSK Datakeeper: Pertec hard disk controller
`icom`	iCOM FD3712/FD3812: 8" floppy controller with boot PROM
`mds`	MITS 88-MDS: 5.25" minidisk controller
`mdsa`	North Star MDS-A: single-density 5.25" floppy controller
`mdsad`	North Star MDS-A-D: double-density 5.25" floppy controller
`tarbell`	Tarbell #1011: single-density floppy controller
`tarbelldd`	Tarbell #2022: double-density floppy controller
`versafloppy`	SD Systems VersaFloppy I/II: WD177x floppy controller

Serial

Type	What it is
`2sio`	MITS 88-2SIO: two 6850 serial ports
`gsio`	Generic SIO: two strap-configurable serial channels
`io4`	SSM IO-4: two serial + two parallel ports
`pmmi`	PMMI MM-103: Bell 103 modem
`propio`	S100Computers Console IO: Propeller console serial port
`sbc`	SD Systems SBC-100/200: Z80 single-board computer
`sio`	MITS 88-SIO: one serial port (COM2502 UART)
`turnkey`	MITS 8800b Turnkey Module: boot PROM, serial, auto-start

Tape

Type	What it is
`acr`	MITS 88-ACR: audio cassette interface
`uio`	MITS 88-UIO: serial port + cassette interface

Parallel and printer

Type	What it is
`4pio`	MITS 88-4PIO: up to four 6820 parallel ports
`c700`	MITS 88-C700: Centronics line-printer controller
`d7a`	Cromemco D+7A: analog + parallel I/O, joysticks
`lpc`	MITS 88-LPC: 88-LP line-printer controller
`music6`	Newtech Model 6 Music Board: 6-bit D/A and speaker
`pio`	MITS 88-PIO: 8-bit parallel port

Video

Type	What it is
`cadzilla`	CADzilla: HD63484 ACRTC graphics board with a Bt453 RAMDAC
`dazzler`	Cromemco Dazzler: color graphics
`dazzler2`	Cromemco Dazzler II: color graphics, on-board RAM
`vdb8024`	SD Systems VDB-8024: 80x24 video terminal board
`vdm1`	Processor Technology VDM-1: 16x64 memory-mapped video

Systems

Type	What it is
`sol`	Processor Technology Sol-PC I/O: serial, keyboard, tape

PROM programmer

Type	What it is
`pb1`	SSM PB1: 2708/2716 EPROM programmer

Other

Type	What it is
`fp`	Altair front panel: the SENSE switches at IN 0FFH
`hostbridge`	Host Bridge: file transfer to the host (not a period board)
`rtc100`	SciTronics RTC-100: battery-backed clock/calendar
`ss1`	CompuPro System Support 1: interrupts, timer, clock, serial
`virtc`	MITS 88-VI/RTC: vectored interrupts + real-time clock

Machines

Machine	What it is
`8085`	A minimal 8085 machine: an `8085` CPU, 64K of RAM, and a 2SIO console.
`acuter`	ACUTER at F000 -- CUTER on a plain Altair, with a terminal instead of a VDM.
`altmon`	An Altair with a monitor in ROM and a terminal on it.
`amon`	AMON 3.1 in a 4K EPROM at F000 -- Martin Eberhard's full-featured Altair monitor.
`bankmem`	A bank-switched RAM machine: a Z80, a console, and a Vector Graphic 64K bankmem.
`basic4k`	The machine Altair 4K BASIC was sold to run on: an 88-SIO Teletype, a cassette in the ACR.
`basic8k`	The machine Altair 8K BASIC was sold to run on: an 88-2SIO terminal, a cassette in the ACR.
`cadzilla`	An HD63484 ACRTC graphics board with a Bt453 RAMDAC in an Altair -- the bench for CADzilla.
`cdbl`	The `default` machine with the Combo Disk Boot Loader in the PROM socket.
`compupro`	A stock Altair with a CompuPro System Support 1 board for its clock/calendar.
`cuter`	CUTER 1.3 driving a Processor Technology VDM-1 -- the real Sol/CUTS monitor.
`dazzler`	A Cromemco Dazzler in an Altair -- the bench for the S-100's first color graphics card.
`default`	The machine you get when you name none: 56K, and the DBL boot PROM at FF00.
`dualide`	S100Computers "IDE-AB CF" machine -- a V2 Z80 CPU board booting CP/M 3 off a CompactFlash card.
`dualidesd`	S100Computers "IDE-AB CF+ESP32" machine -- both boards, CP/M 3 on CF drives A:/B: and SD drives C:/D:.
`dualsd`	S100Computers "Dual SD" machine -- a V2 Z80 CPU board booting CP/M 3 off a microSD card.
`icom`	iCOM FD3712 8" floppy machine -- boots CP/M 2.2 off a single-density iCOM disk.
`lineprinter-lpc`	The `default` machine with an 88-LPC line printer at port 02; CONNECT `lpt0:prn` to a file or the console.
`lineprinter`	The `default` machine with an 88-C700 line printer at port 02; CONNECT `lpt0:prn` to a file or the console.
`minidisk`	The Altair Minidisk: an 88-MDS at 08 and the MDBL boot PROM. You supply the 5.25" disk.
`northstar`	North Star MDS-A single-density floppy: the controller at E800 and its boot PROM. You supply the disk.
`northstardd`	North Star MDS-A-D double-density floppy: the controller at E800 and its boot PROM. You supply the disk.
`original`	The Altair as it actually left Albuquerque.

`parallel`	The `default` machine with two MITS parallel boards: an 88-PIO and an 88-4PIO.
`ps2`	The machine MITS Programming System II ran on: basic8k's cards, but not basic8k's bootstrap.
`ps2int`	MITS Programming System II, WITH INTERRUPTS. `ps2` with A9 down and an 88-VI/RTC in it.
`rombasic`	MITS Extended ROM BASIC 16K -- Extended BASIC that runs directly out of ROM.
`sbc200`	SD Systems SBC-200 -- a 4 MHz Z80 single-board computer running the MSMONR21 monitor.
`sbc200v`	The SD Systems SBC-200 with a VDB-8024 VIDEO console: SDMONV21 on an 80x24 screen.
`sol20`	The Processor Technology Sol-20 -- an integrated 8080 machine, running SOLOS.
`tarbell`	Tarbell #1011 single-density floppy machine -- boots CP/M 2.2 the moment a disk is in it.
`tarbelldd`	Tarbell #2022 double-density floppy machine -- boots CP/M 2.2 the moment a disk is in it.
`turnkey`	The MITS 8800bt -- an Altair with a Turnkey Module where the front panel used to be.
`vdm1`	A Processor Technology VDM-1 in an Altair, and a demo that draws on it.
`z80`	A minimal Z80 machine: a `z80` CPU, 64K of RAM, and a 2SIO console.

A machine file, in one look

```
[machine]
name    = "mine"
base    = "default"        # start from a machine, and say what is DIFFERENT
startup = ["RUN FF00"]     # the operator's own keystrokes. There is no BOOT verb.

[[board]]                  # type + a NEW id      -> ADD the card
type = "2sio"              # type + an id from the base -> REPLACE it outright
id   = "sio0"              # NO type + an id      -> MODIFY the one already there
port = 10                  # remove = true        -> PULL THE CARD OUT

  [board.unit.a]           # a unit's own settings
  connect = "console"

  [[board.region]]         # a list the card owns (memory)
  type = "ram"
  at   = 0000              # hex: it is an address
  size = "56K"             # decimal: it is a size

  [[board.drive]]          # a list the card owns (disk controllers)
  unit  = 0
  mount = "cpm.dsk"        # relative to THIS FILE

[console]                  # the HOST's terminal -- not a board
strip7out = true
base      = octal          # read/print the wire class in split octal (MITS style)
```

Paths: a path *inside* a machine file is relative to **that file**. A path you *type* is relative to **your shell**.

Endpoints — `CONNECT <id>:<unit> <endpoint>`

Endpoint	Is
`console`	the host terminal. Exactly one unit may hold it.
`null`	nowhere. Writes vanish, reads never come.
`loopback`	itself — what you write comes back.
`scripted`	a caller in place of a human — what MCP and the tests type into.
`socket:PORT`	**listens**, as a raw pipe — for a program at the far end. `?banner` greets each caller.
`socket:HOST:PORT`	**calls out**, as a raw pipe.
`telnet:PORT`	**listens**, speaking Telnet — this is telnet-in for a person. Greets each caller; `?banner=off` stops it.
`telnet:HOST:PORT`	**calls out**, taking the telnet client's part.
`serial:DEVICE`	a real serial port on this host.
`pty`	a pseudo-terminal (macOS and Linux): open `/tmp/altairsim0` with a terminal program. `pty:LINK` selects the name.
`in:PATH`	a host file as a reader (paper tape). `?cps=N` paces it.
`out:PATH`	a host file as a punch — 8-bit clean, never truncating.
`terminal`	a window the simulator draws itself (SDL builds). `?emulation=vt100|adm3a|vt52|h19`, `?size=COLSxROWS`.
`printer:QUEUE`	a real print queue on this host.
`<endpoint>|FILE`	a tap: append `|FILE` to any endpoint to also log the line. `?fmt=text` = plain text, only what the guest prints.
`<endpoint>|socket:PORT`	a live mirror: `telnet` in to watch and take over. `?ro` = watch-only.
`<endpoint>|pty[:LINK]`	the same mirror on a pseudo-terminal (macOS and Linux): open `/tmp/altairsim0`, or `LINK`, with a terminal program.
`<endpoint>|serial:DEVICE`	the same mirror on a real serial port, 8N1: a hardware terminal watches and types. `?baud=N` (default 9600), `?ro`.